

Отчёт по лабораторной работе №24

по курсу «Алгоритмы и структуры данных».

Выполнил студент группы М8О-114БВ-25 Богатырева Екатерина Денисовна № по списку 2.

Контакты: bogatyrevaK2019@yandex.ru

Работа выполнена: «20» апреля 2026 г.

Преподаватель: каф.806 Сысоев Максим Алексеевич

Входной контроль знаний с оценкой:

Отчет сдан «21» апреля 2026 г.

Итоговая оценка: _____

Подпись преподавателя: _____

1. **Тема:** Динамические структуры данных. Обработка деревьев. Трансформация арифметических выражений.

2. **Цель работы:** Составить программу на языке Си для представления арифметических выражений в виде бинарного дерева, выполнения заданного преобразования (вариант 2) и вывода результата в виде дерева и в текстовом виде.

3. **Задание (Вариант № 2):** Упростить выражения, выполнив вычитание: $3 - 5 \rightarrow -2$, $5 + b - 3 \rightarrow 2 + b$

4. **Оборудование:** *Оборудование ПЭВМ студента, если использовалось:*

Процессор: Intel(R) Pentium(R) CPU 6405U @ 2.40GHz 2.40 GHz, ОП 8 ГБ, SSD 256 ГБ

5. **Программное обеспечение:** *Программное обеспечение ЭВМ студента, если использовалось:*
Операционная система семейства Unix, наименование Ubuntu, версия 24.04 LTS, интерпретатор команд `bash` версия 5.2.21.

Система программирования: gcc (Ubuntu 11.4.0-1ubuntu1~22.04.2) версия 11.4 Редактор текстов: VS code версия: 1.107.1.

Язык программирования: Си

6. **Идея, метод, алгоритм решения задачи** [в формах: словесной, псевдокода, графической (блок-схема, диаграмма, рисунок, таблица) или формальные спецификации с пред- и постусловиями]:

Идея: Арифметическое выражение представляется в виде бинарного дерева. Листья — операнды (числа или переменные), внутренние узлы — операции. Преобразование выполняется обходом дерева: если оба операнда — числа, операция выполняется сразу, и узел заменяется на результат.

Метод:

1. Лексический анализ входной строки (разбиение на токены)
2. Преобразование инфиксной формы в постфиксную (алгоритм Дейкстры)
3. Построение дерева выражений из постфиксной записи
4. Рекурсивное преобразование дерева: замена узлов число+число и число-число на результат
5. Вывод результата в инфиксной форме и в виде дерева

Алгоритм:

Начало

Ввод выражения

Лексический анализ $\rightarrow$ массив токенов

Алгоритм Дейкстры $\rightarrow$ постфиксная форма

Построение дерева

Вывод исходного выражения и дерева

Рекурсивное преобразование:

- Если узел = число+число → вычислить
- Если узел = число-число → вычислить
- Иначе рекурсивно обойти детей

Вывод преобразованного выражения и дерева

Освобождение памяти

Конец

Сценарий выполнения работы (план работы, первоначальный текст программы в черновике (можно на отдельном листе) и тесты либо соображения по тестированию). **План работы:**

1. **Изучение теории** – алгоритмы разбора выражений, постфиксная форма, бинарные деревья
2. **Реализация лексического анализатора** (analyzer) – разбиение строки на токены
3. **Реализация алгоритма сортировочной станции** (to_postfix) – инфикс → постфикс
4. **Реализация построения дерева** (buildTree) – из постфиксной записи
5. **Реализация преобразования** (transform) – вычисление числовых операций
6. **Реализация вывода** (printInfix, printTree) – инфиксная форма и дерево
7. **Тестирование** на нескольких выражениях

Тесты:

3-5 -2
10+20 30
5+b-3 2+b
a+7-2+b-1 4+a+b

Соображения по тестированию:

1. Проверить выражения без скобок и со скобками.
2. Проверить приоритет операций (* и / выше + и -).
3. Проверить корректную обработку унарного минуса в начале выражения и после других операторов.
4. Проверить вложенные вычитания.
5. Убедиться, что дерево после преобразования сохраняет семантику исходного выражения.

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

Допущен к выполнению работы. Подпись преподавателя _____

7. Распечатка протокола (подклеить листинг окончательного варианта программы с тестовыми примерами, подписанный преподавателем). #include <ctype.h>

```
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum Type {
    NUMBER,
    OPERATOR,
    LPAREN,
    RPAREN
};

typedef struct REQ {
    enum Type type;
    char text[100];
    char op;
} REQ;

typedef struct Node {
    char op;
    char *value;
    struct Node *left;
    struct Node *right;
```

```
} Node;
```

```
Node *createValue(const char *v) {  
    Node *newNode = calloc(1, sizeof(Node));  
    if (newNode == NULL) return NULL;  
    newNode->value = (char *)malloc(strlen(v) + 1);  
    if (newNode->value == NULL) {  
        free(newNode);  
        return NULL;  
    }  
    strcpy(newNode->value, v);  
    return newNode;  
}
```

```
Node *createOp(char op, Node *left, Node *right) {  
    Node *newNode = calloc(1, sizeof(Node));  
    if (newNode == NULL) return NULL;  
    newNode->op = op;  
    newNode->left = left;  
    newNode->right = right;  
    return newNode;  
}
```

```
int is_op(char c) {  
    return c == '+' || c == '-' || c == '*' || c == '/';  
}
```

```
int order(char op) {  
    if (op == '+' || op == '-') return 1;  
    if (op == '*' || op == '/') return 2;  
    if (op == '~') return 3;  
    return 0;  
}
```

```
int analyzer(const char *s, REQ *arr, int *n) {  
    int k = 0;  
    for (int i = 0; s[i] != '\0' && s[i] != '\n';) {  
        if (isspace(s[i])) {  
            i++;  
            continue;  
        }  
        if (isdigit(s[i])) {  
            int start = i;  
            i++;  
            while (isdigit(s[i])) i++;  
            int length = i - start;  
            memcpy(arr[k].text, s + start, length);  
            arr[k].text[length] = '\0';  
            arr[k].type = NUMBER;  
            k++;  
            continue;  
        }  
        if (isalpha(s[i])) {  
            int start = i;  
            i++;  
            while (isalpha(s[i])) i++;
```

```

        int length = i - start;
        memcpy(arr[k].text, s + start, length);
        arr[k].text[length] = '\0';
        arr[k].type = NUMBER;
        k++;
        continue;
    }
    if (s[i] == '+' || s[i] == '-' || s[i] == '*' || s[i] == '/') {
        arr[k].type = OPERATOR;
        arr[k].op = s[i];
        arr[k].text[0] = s[i];
        arr[k].text[1] = '\0';
        k++;
        i++;
        continue;
    }
    if (s[i] == '(') {
        arr[k].type = LPAREN;
        k++;
        i++;
        continue;
    }
    if (s[i] == ')') {
        arr[k].type = RPAREN;
        k++;
        i++;
        continue;
    }
    printf("Неправильный ввод\n");
    return 0;
}
*n = k;
return 1;
}

int to_postfix(const REQ *in, int in_n, REQ *out, int *out_n) {
    REQ stack[1000];
    int top = -1;
    int j = 0;
    enum Type prev = OPERATOR;
    for (int i = 0; i < in_n; i++) {
        REQ cur = in[i];
        if (cur.type == NUMBER) {
            out[j++] = cur;
            prev = NUMBER;
            continue;
        }
        if (cur.type == LPAREN) {
            stack[++top] = cur;
            prev = LPAREN;
            continue;
        }
        if (cur.type == RPAREN) {
            while (top >= 0 && stack[top].type != LPAREN) {
                out[j++] = stack[top--];
            }

```

```

        if (top < 0) return 0;
        top--;
        prev = RPAREN;
        continue;
    }
    if (cur.type == OPERATOR) {
        if (cur.op == '-' && (i == 0 || prev == OPERATOR || prev == LPAREN)) {
            cur.op = '~';
        }
        while (top >= 0 && stack[top].type == OPERATOR) {
            int p1 = order(stack[top].op);
            int p2 = order(cur.op);
            if ((cur.op == '~' && p2 < p1) || (cur.op != '~' && p2 <= p1)) {
                out[j++] = stack[top--];
            } else {
                break;
            }
        }
        stack[++top] = cur;
        prev = OPERATOR;
    }
}
while (top >= 0) {
    if (stack[top].type != OPERATOR) return 0;
    out[j++] = stack[top--];
}
*out_n = j;
return 1;
}

```

```

Node *buildTree(const REQ *pf, int n) {
    Node *stack[1000];
    int top = -1;
    for (int i = 0; i < n; i++) {
        if (pf[i].type == NUMBER) {
            Node *leaf = createValue(pf[i].text);
            stack[++top] = leaf;
            continue;
        }
        if (pf[i].type == OPERATOR && pf[i].op == '~') {
            Node *right = stack[top--];
            Node *node = createOp('~', NULL, right);
            stack[++top] = node;
            continue;
        }
        if (pf[i].type == OPERATOR) {
            Node *right = stack[top--];
            Node *left = stack[top--];
            Node *node = createOp(pf[i].op, left, right);
            stack[++top] = node;
        }
    }
    return stack[0];
}

```

```

int isNumber(Node *node) {

```

```

    if (node == NULL) return 0;
    if (node->value == NULL) return 0;
    for (int i = 0; node->value[i]; i++) {
        if (!isdigit(node->value[i])) return 0;
    }
    return 1;
}

```

```

int isVariable(Node *node) {
    if (node == NULL) return 0;
    if (node->value == NULL) return 0;
    for (int i = 0; node->value[i]; i++) {
        if (!isalpha(node->value[i])) return 0;
    }
    return 1;
}

```

```

Node *transform(Node *node) {
    if (node == NULL) return NULL;

    node->left = transform(node->left);
    node->right = transform(node->right);

    if (node->value != NULL) return node;

```

```

    if (node->op == '+' && isNumber(node->left) && isNumber(node->right)) {
        int result = atoi(node->left->value) + atoi(node->right->value);
        char buf[32];
        sprintf(buf, "%d", result);
        Node *newNode = createValue(buf);
        freeTree(node->left);
        freeTree(node->right);
        free(node);
        return newNode;
    }

```

```

    if (node->op == '-' && isNumber(node->left) && isNumber(node->right)) {
        int result = atoi(node->left->value) - atoi(node->right->value);
        char buf[32];
        sprintf(buf, "%d", result);
        Node *newNode = createValue(buf);
        freeTree(node->left);
        freeTree(node->right);
        free(node);
        return newNode;
    }

```

```

    return node;
}

```

```

void printInfix(const Node *node) {
    if (node == NULL) return;
    if (node->value != NULL) {
        printf("%s", node->value);
        return;
    }
}

```

```

    if (node->op == '~') {
        printf("-");
        printInfix(node->right);
        printf(")");
        return;
    }
    printf("(");
    printInfix(node->left);
    printf(" %c ", node->op);
    printInfix(node->right);
    printf(")");
}

void printTree(const Node *node, int d) {
    if (node == NULL) return;
    printTree(node->right, d + 1);
    for (int i = 0; i < d; i++) {
        printf(" ");
    }
    if (node->value != NULL) {
        printf("%s\n", node->value);
    } else if (node->op == '~') {
        printf("~\n");
    } else {
        printf("%c\n", node->op);
    }
    printTree(node->left, d + 1);
}

void freeTree(Node *node) {
    if (node == NULL) return;
    freeTree(node->left);
    freeTree(node->right);
    free(node->value);
    free(node);
}

int main() {
    system("chcp 65001");
    char input[1000];
    REQ arr[1000], postfix[1000];
    int tn = 0, pn = 0;

    printf("Пример: 3 - 5 -> -2\n");
    printf("Пример: 5 + b - 3 -> 2 + b\n");
    printf("Введите выражение: ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        fprintf(stderr, "Неправильный ввод\n");
        return 1;
    }

    if (!analyzer(input, arr, &tn) || !to_postfix(arr, tn, postfix, &pn)) {
        return 1;
    }
}

```

```

Node *root = buildTree(postfix, pn);
if (root == NULL) {
    return 1;
}

printf("\nДо:\n");
printInfix(root);
printf("\n\nДерево до:\n");
printTree(root, 0);

root = transform(root);

printf("\nПосле:\n");
printInfix(root);
printf("\n\nДерево после:\n");
printTree(root, 0);

freeTree(root);
return 0;
}

```

Дневник отладки должен содержать дату и время сеансов отладки и основные события (ошибки в сценарии и программе, нестандартные ситуации) и краткие комментарии к ним. В дневнике отладки приводятся сведения об использовании других ЭВМ, существенном участии преподавателя и других лиц в написании и отладке программы.

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

8. **Замечания автора** по существу работы: замечания отсутствуют.

9. **Выводы:** В ходе выполнения лабораторной работы разработана программа на языке Си, реализующая упрощение арифметических выражений с использованием бинарных деревьев. Реализованы лексический анализ, алгоритм сортировочной станции, построение дерева, рекурсивное преобразование для вычисления числовых операций (сложение и вычитание чисел), а также вывод инфиксной записи и визуализация дерева. Программа успешно вычисляет выражения, состоящие только из чисел (например, $3-5 \rightarrow -2$). Работа позволила закрепить навыки работы с динамическими структурами данных, рекурсивными алгоритмами и обработкой выражений.

Подпись студента _